



UNIVERSITÉ DE KINSHASA
FACULTÉ DES SCIENCES ET TECHNOLOGIES
DÉPARTEMENT DE MATHÉMATIQUES, STATISTIQUE ET
INFORMATIQUE

Mention Informatique, M1 LMD
Filière Data Science et Intelligence Artificielle

TRAVAIL DU COURS DE DATA SCIENCE ET BIG DATA

SUJET :
« ALGORITHME DE NUÉES
DYNAMIQUES »

Présenté par :
BUETUENA MALOZI JORDI

Encadré par :
Professeur KAFUNDA Pierre
Doctorant KAMINGU Gradi

Année Académique : 2025-2026

Table des matières

1	Introduction sur le Machine Learning	2
2	Algorithme de Nuées Dynamiques	3
2.1	Approche Mathématique	3
2.2	Implémentation en Python (From Scratch)	4
3	Applications	9
4	Forces et Faiblesses	10
4.1	Forces	10
4.2	Faiblesses	10
5	Conclusion	11
6	Références Bibliographiques	12
6.1	Annexe	13
6.1.1	Illustration de l'exécution	13

Chapitre 1

Introduction sur le Machine Learning

Le Machine Learning (ou apprentissage automatique) est une sous-discipline de l'Intelligence Artificielle qui permet aux systèmes d'apprendre à partir de données sans être explicitement programmés pour chaque tâche. On le divise généralement en deux grandes familles :

- **L'apprentissage supervisé** : L'algorithme apprend à partir d'un ensemble de données étiquetées (ex : régression, classification).
- **L'apprentissage non supervisé** : L'algorithme explore des données non étiquetées pour y découvrir des structures cachées, des regroupements ou des anomalies.
- **L'apprentissage par renforcement** : Apprendre par essais, erreurs et récompenses. Un agent IA apprend à accomplir une tâche en interagissant avec son environnement. S'il fait une bonne action, il reçoit une récompense ; s'il se trompe, une pénalité. L'objectif est d'accumuler le maximum de points.
- **L'apprentissage semi-supervisé** : Entraîner une IA avec des données entièrement annotées par l'humain coûte très cher. Ce mode utilise une petite quantité de données étiquetées combinée à une grande quantité de données non étiquetées. L'IA utilise le peu d'exemples connus pour deviner la structure du reste.
- **L'apprentissage par transfert** : On prend un modèle IA déjà entraîné sur un problème général et très vaste, puis on le réutilise (on le réadapte) pour une tâche spécifique nécessitant beaucoup moins de données.
- **L'apprentissage fédéré** : Au lieu d'envoyer toutes les données personnelles sur un serveur central, le modèle d'IA est envoyé sur les appareils des utilisateurs (smartphones, hôpitaux). Chaque appareil entraîne l'IA localement, puis renvoie uniquement les améliorations du modèle (pas vos données) au serveur central pour les fusionner.

C'est précisément dans cette seconde catégorie que s'inscrivent les méthodes de **classification non supervisée** (ou *clustering*), dont l'algorithme des nuées dynamiques est l'un des représentants historiques les plus puissants. L'objectif est de partitionner un ensemble d'individus en K groupes homogènes (les classes ou "nuées") de sorte que les individus d'un même groupe soient très ressemblants, tandis que ceux de groupes différents soient le plus éloignés possible.

Chapitre 2

Algorithme de Nuées Dynamiques

2.1 Approche Mathématique

L'algorithme des nuées dynamiques (variante itérative proche des K -Means) repose sur la minimisation d'un critère d'inertie intra-classe.

Soit un ensemble de données $X = \{x_1, x_2, \dots, x_N\}$ composé de N observations, où chaque observation x_i est un vecteur de $\mathbb{R}^d$ (espace à d dimensions). On cherche à partitionner X en K classes disjointes $C_1, C_2, \dots, C_K$ représentées chacune par un centre de gravité (ou noyau) g_k .

1. **Le Centre de Gravité d'une classe C_k** : Il est défini par la moyenne des vecteurs appartenant à cette classe :

$$g_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

où $|C_k|$ est le nombre d'éléments dans la classe C_k .

2. **La Règle d'Affectation (Étape de Classification)** : Chaque individu x_i est affecté à la classe C_k dont le centre g_k est le plus proche (généralement au sens de la distance euclidienne) :

$$x_i \in C_k \iff d(x_i, g_k) \leq d(x_i, g_j), \quad \forall j \in \{1, \dots, K\}$$

3. **Le Critère d'Optimisation (Inertie Intra-Classe W)** : L'algorithme cherche à minimiser l'inertie intra-classe totale, qui mesure la compacité des groupes :

$$W = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - g_k\|^2$$

Déroulement de l'algorithme :

- **Étape 1** : Initialisation aléatoire des K centres $g_1, g_2, \dots, g_K$.
- **Étape 2** : Affectation de chaque point au centre le plus proche (création des partitions).
- **Étape 3** : Mise à jour des centres en recalculant le barycentre de chaque nouvelle classe.
- **Étape 4** : Répétition des étapes 2 et 3 jusqu'à **convergence** (lorsque les centres ne bougent plus ou que les affectations des points restent stables).

2.2 Implémentation en Python (From Scratch)

Voici un code simple et commenté, n'utilisant que les bibliothèques fondamentales (numpy et matplotlib).

```
1 #Algorithme implémenté par BUETUENA MALOZI JORDI, étudiant en
   Master 1 LMD, Data Science & IA; l'Université de Kinshasa
2 #coding:utf-8
3 import math
4 import random
5
6 #Fonctions de distance
7 def distance_euclidienne(a, b):
8     return math.sqrt(sum((ai - bi) ** 2 for ai, bi in zip(a, b)))
9
10 def distance_manhattan(a, b):
11     return sum(abs(ai - bi) for ai, bi in zip(a, b))
12
13 #Prototypes
14 def moyenne(points):
15     n = len(points)
16     dim = len(points[0])
17     return [sum(p[d] for p in points) / n for d in range(dim)]
18
19 def mediane(points):
20     dim = len(points[0])
21     return [
22         sorted(p[d] for p in points)[len(points) // 2]
23         for d in range(dim)
24     ]
25
26 #Algorithme des nuées dynamiques
27 class NueesDynamiques:
28     def __init__(
29         self,
30         k=3,
31         max_iter=300, #Nombre d'itérations maximales
32         tol=1e-6,
33         random_state=None,
34         distance=distance_euclidienne,
35         prototype_update=moyenne,
36     ):
37         self.k = k
38         self.max_iter = max_iter
39         self.tol = tol
40         self.random_state = random_state
41         self.distance = distance
42         self.prototype_update = prototype_update
43
44         self.prototypes_ = None
45         self.labels_ = None
46         self.critere_ = None
```

```

47
48 def _initialiser(self, X):
49     rng = random.Random(self.random_state)
50     indices = rng.sample(range(len(X)), self.k)
51     return [list(X[i]) for i in indices]
52
53 def fit(self, X):
54     X = [list(x) for x in X]
55     n = len(X)
56
57     if n < self.k:
58         raise ValueError("k doit tre  <= au nombre de points.")
59
60     self.prototypes_ = self._initialiser(X)
61     labels = [-1] * n
62
63     for _ in range(self.max_iter):
64         nouveaux_labels = []
65         for x in X:
66             distances = [self.distance(x, p) for p in self.prototypes_]
67             j = min(range(self.k), key=distances.__getitem__)
68             nouveaux_labels.append(j)
69
70         nouveaux_prototypes = []
71         for j in range(self.k):
72             cluster = [X[i] for i in range(n) if nouveaux_labels[i] == j]
73             if cluster:
74                 nouveaux_prototypes.append(self.prototype_update(cluster))
75             else:
76                 i_far = max(
77                     range(n),
78                     key=lambda i: self.distance(
79                         X[i], self.prototypes_[nouveaux_labels[i]]
80                     ),
81                 )
82                 nouveaux_prototypes.append(list(X[i_far]))
83
84         deplacement = sum(
85             self.distance(self.prototypes_[j], nouveaux_prototypes[j])
86             for j in range(self.k)
87         )
88
89         self.prototypes_ = nouveaux_prototypes
90
91         if nouveaux_labels == labels and deplacement <= self.tol:
92             labels = nouveaux_labels
93             break
94
95         labels = nouveaux_labels
96
97     self.labels_ = labels

```

```

98 self.critere_ = sum(
99 self.distance(X[i], self.prototypes_[labels[i]]) for i in range(n)
100 )
101 return self
102
103 def fit_predict(self, X):
104 return self.fit(X).labels_
105
106 #Saisie des valeurs par l'utilisateur
107
108 def demander_entier(message, minimum=1):
109 while True:
110 try:
111 v = int(input(message))
112 if v < minimum:
113 print(f"Valeur >= {minimum} attendue.")
114 continue
115 return v
116 except ValueError:
117 print("Entier invalide, r essayez.")
118
119 def demander_flottant(message):
120 while True:
121 try:
122 return float(input(message))
123 except ValueError:
124 print("Nombre invalide, r essayez.")
125
126 def saisir_points():
127 n = demander_entier("Nombre de points saisir : ", minimum=1)
128 dim = demander_entier("Nombre de dimensions (ex: 2 pour (x,y)) : ",
129 minimum=1)
130
131 X = []
132 print(f"\nSaisie des {n} points ({dim} coordonn es chacun) :")
133 for i in range(n):
134 print(f"\nPoint {i + 1}")
135 point = [demander_flottant(f" Coordonn e {d + 1} : ") for d in
136 range(dim)]
137 X.append(point)
138 return X
139
140 def choisir_distance():
141 print("\nChoix de la distance :")
142 print(" 1. Euclidienne (prototype = moyenne) -> k-means
143 classique")
144 print(" 2. Manhattan (prototype = m diane) -> k-medians")
145 while True:
146 c = input("Votre choix (1/2) [1] : ").strip() or "1"
147 if c == "1":
148 return distance_euclidienne, moyenne, "Euclidienne / moyenne"

```

```

146 if c == "2":
147     return distance_manhattan, mediane, "Manhattan / m diane"
148 print("Choix invalide.")
149
150 #Programme principal
151 def main():
152     print("    ALGORITHME DES NUÉES DYNAMIQUES ")
153     X = saisir_points()
154     k = demander_entier(
155         f"\nNombre de clusters k (2 <= k <= {len(X)}) : ", minimum=2
156     )
157     while k > len(X):
158         print(f"k doit tre <= {len(X)}.")
159     k = demander_entier(f"Nombre de clusters k : ", minimum=2)
160
161     dist_fn, proto_fn, nom_methode = choisir_distance()
162
163     max_iter = demander_entier(
164         "\nNombre max d'it rations [par d faut 300] : ", minimum=1
165     )
166
167     graine = input("Graine al atoire (vide = al atoire) : ").strip()
168     random_state = int(graine) if graine else None
169
170     print("\n\n")
171     modele = NueesDynamiques(
172         k=k,
173         max_iter=max_iter,
174         random_state=random_state,
175         distance=dist_fn,
176         prototype_update=proto_fn,
177     )
178     modele.fit(X)
179
180     print(f"M thode : {nom_methode}")
181
182     for j in range(k):
183         membres = [i for i, lab in enumerate(modele.labels_) if lab == j]
184         print(f"\nCluster {j + 1} (prototype = {modele.prototypes_[j]})")
185         for i in membres:
186             print(f"    Point {i + 1} : {X[i]}")
187
188     print("\nR capitulatif")
189     print("Labels      :", modele.labels_)
190     print("Prototypes    :", modele.prototypes_)
191     print(f"Crit re total: {modele.critere_:.6f}")
192
193 if __name__ == "__main__":
194     main()

```

Listing 2.1 – Implémentation de l'algorithme des nuées dynamiques from scratch

Chapitre 3

Applications

Les nuées dynamiques et les algorithmes de partitionnement apparentés possèdent de vastes domaines d'application en Data Science :

- **Segmentation de clientèle** : Regrouper des clients selon leurs comportements d'achat pour cibler des campagnes marketing.
- **Compression d'images et Réduction de couleurs** : Réduire le nombre de couleurs d'une image en regroupant les pixels similaires (quantification vectorielle).
- **Compression de données et Systèmes de Recommandation** : Identifier des profils d'utilisateurs similaires.
- **Prétraitement pour le Big Data** : Servir d'étape préliminaire (*feature engineering*) avant des analyses plus complexes ou pour résumer des flux de données massifs.

Chapitre 4

Forces et Faiblesses

4.1 Forces

- **Simplicité et Intuitivité** : Facile à comprendre, à interpréter mathématiquement et à implémenter.
- **Rapidité d'exécution** : Sa complexité temporelle est généralement linéaire $O(K \cdot N \cdot d)$ par itération, ce qui le rend adapté à de grands volumes de données.
- **Efficacité** : Fonctionne extrêmement bien sur des données sphériques et bien séparées.

4.2 Faiblesses

- **Sensibilité à l'initialisation** : Le choix aléatoire des centres de départ peut conduire à des optima locaux.
- **Fixation du paramètre K** : L'utilisateur doit fixer à l'avance le nombre de groupes K .
- **Sensibilité aux valeurs aberrantes (*outliers*)** : Les points très éloignés faussent le calcul des moyennes (barycentres).

Chapitre 5

Conclusion

L'algorithme des nuées dynamiques demeure un incontournable de la boîte à outils du Data Scientist. Bien qu'il existe des méthodes plus robustes ou adaptées à des structures de données complexes (comme DBSCAN ou le clustering hiérarchique), sa simplicité d'implémentation et son efficacité computationnelle en font une solution de premier choix pour une grande variété de problématiques d'apprentissage non supervisé et de traitement massif de données.

Chapitre 6

Références Bibliographiques

1. Diday, E. (1971). *La méthode des nuées dynamiques*. Revue Française de Informatique et de Recherche Opérationnelle (RO), 5(V1), 61-85.
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning : Data Mining, Inference, and Prediction*. Springer Science & Business Media.
3. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer-Verlag.

6.1 Annexe

6.1.1 Illustration de l'exécution



```
IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: F:/PARCOURS_UNIVERSITAIRE/MASTER 1 LMD INFORMATIQUE/TPS_INTERROGATIONS_EXAMENS/TP DE DATA SCIENCE ET BIG DATA/ALGORITHME_DE_NUEES_DYNAMIQUES.py
ALGORITHME DES NUEES DYNAMIQUES
Nombre de points à saisir : 3
Nombre de dimensions (ex: 2 pour (x,y)) : 2

Saisie des 3 points (2 coordonnées chacun) :

Point 1
Coordonnée 1 : 3.7
Coordonnée 2 : 1.8

Point 2
Coordonnée 1 : 7.9
Coordonnée 2 : 3

Point 3
Coordonnée 1 : 4.43
Coordonnée 2 : 1

Nombre de clusters k (2 <= k <= 3) : 2

Choix de la distance :
1. Euclidienne (prototype = moyenne) -> k-means classique
2. Manhattan (prototype = médiane) -> k-medians
Votre choix (1/2) [1] : 1

Nombre max d'itérations [par défaut 300] : 10
Graine aléatoire (vide = aléatoire) : 13

Méthode : Euclidienne / moyenne

Cluster 1 (prototype = [7.9, 3.0])
• Point 2 : [7.9, 3.0]

Cluster 2 (prototype = [4.0649999999999995, 1.4])
• Point 1 : [3.7, 1.8]
• Point 3 : [4.43, 1.0]

Récapitulatif
Labels : [1, 0, 1]
Prototypes : [[7.9, 3.0], [4.0649999999999995, 1.4]]
Critère total: 1.083005

>>> |
```

FIGURE 6.1 – Illustration de l'exécution